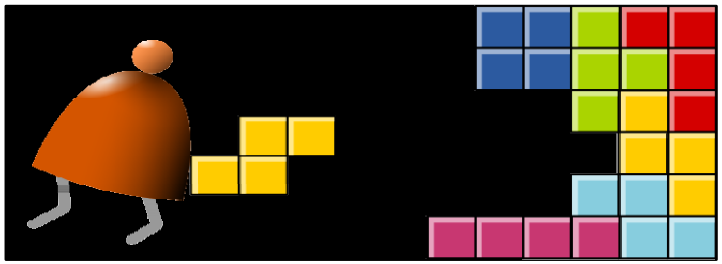
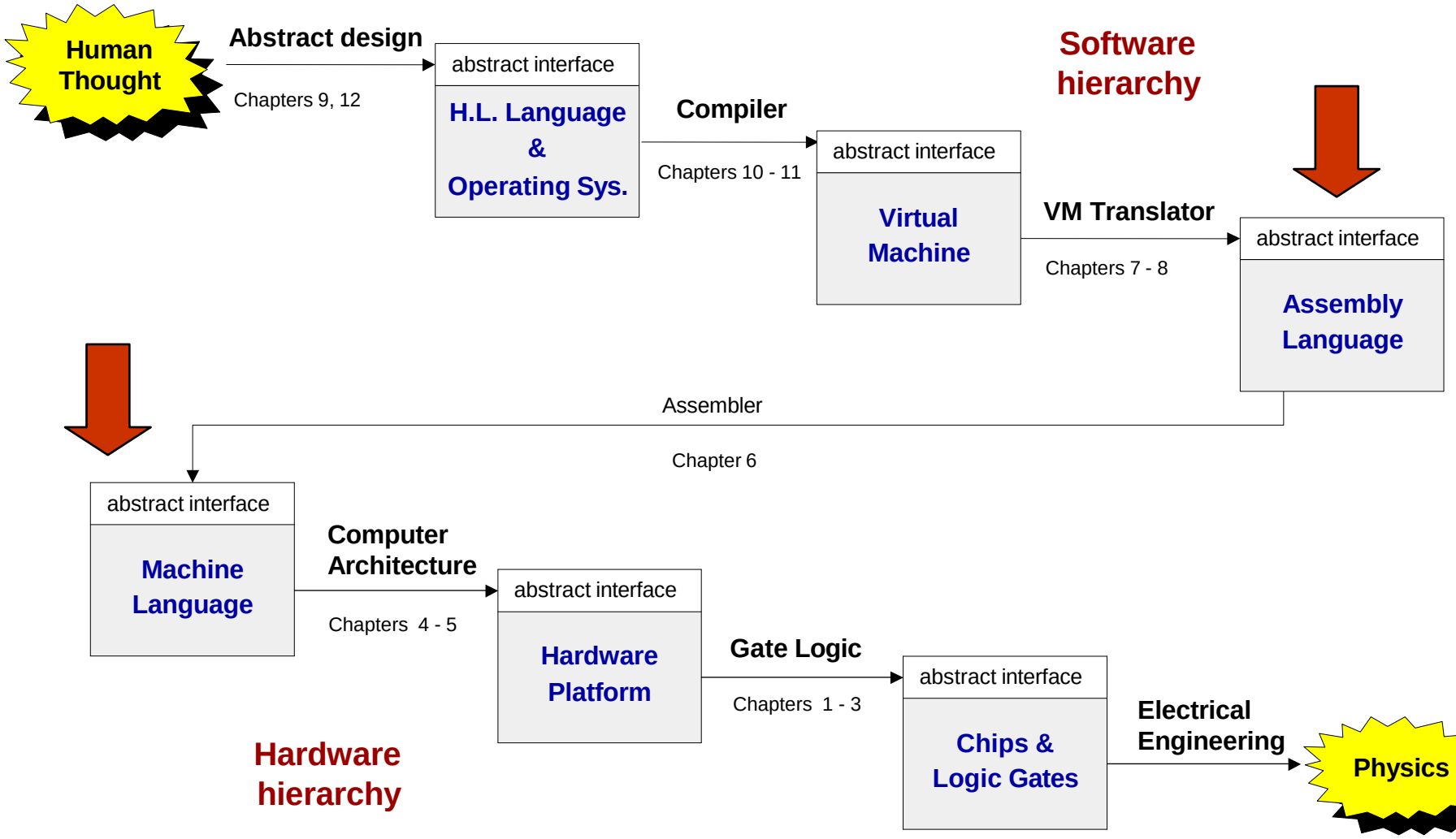
# Machine Language



*Building a Modern Computer From First Principles*

www.nand2tetris.org

# Where we are at:






WOOSONG UNIVERSITY
Kazakhstan

# What is Machine Language?

- Lowest-level programming language a CPU executes directly.

- Sequence of binary-coded instructions (here: 16-bit words).

- Acts as a programmer-oriented abstraction of hardware.



WOOSONG UNIVERSITY
Kazakhstan

# Machine language

**Abstraction – implementation duality:**

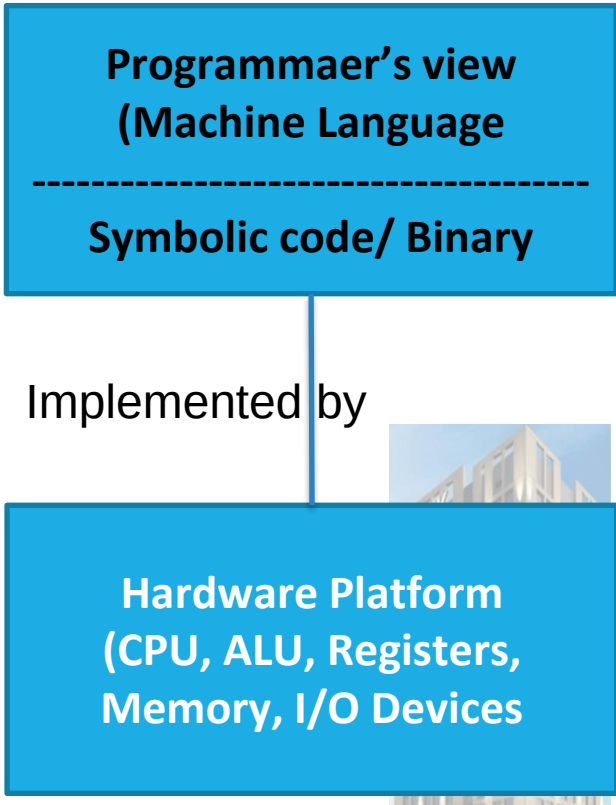
- **Abstraction:** Machine language hides gate-level details from programmers.
- **Implementation:** Hardware (ALU, registers, RAM, PC) physically realizes those instructions.

**Binary vs Symbolic (dual) Representations**

- **Binary version:** Actual **0s** and **1s** executed by hardware.
- **Symbolic version:** Human-readable mnemonics (e.g., ADD R1, R2).

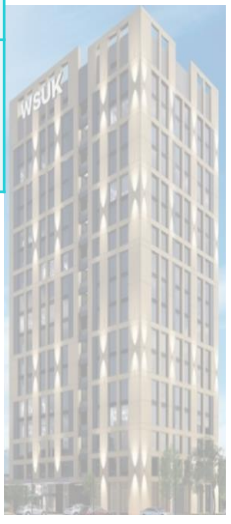
**Loose definition:**

- **Machine language** = a formal system for controlling a processor and memory using a defined set of instructions and registers.
- Same spirit but different syntax across different hardware platforms.



WOOSONG UNIVERSITY
Kazakhstan

| Aspect | Hack Assembly | Hack Machine Language |
|---|---|---|
| **What it is** | Human-readable symbolic form | Binary form executed by the computer |
| **Example** | @17D=M0;JMP | 0000000000010001111111000001000011101010100000111 |
| **Symbols used** | @, D, A, M, labels (LOOP) | Only 0's and 1's |
| **Purpose** | Easier for humans to write and read | Actual code the Hack CPU runs |
| **Conversion** | Translated by an assembler | Directly loaded into ROM and executed |



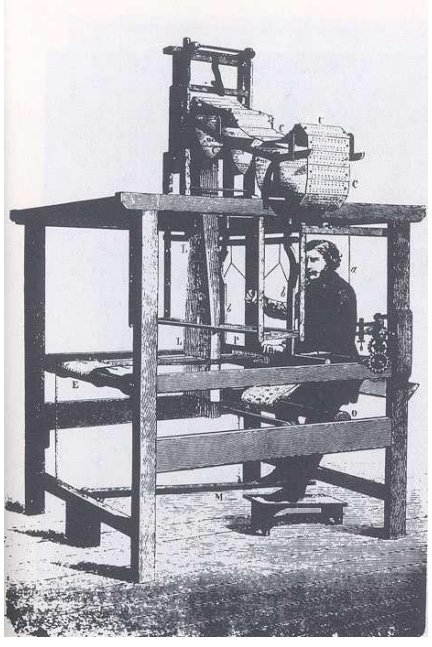
# Binary and symbolic notation

```
1010 0001 0010 1011
```

machine-readable binary form

```
ADD R1, R2, R3
```

human-friendly symbolic form



Jacquard loom
(1801)

Evolution:

- Physical coding
- Symbolic documentation
- Symbolic coding
- Translation and execution
- Requires a *translator*.



Augusta Ada King,
Countess of Lovelace
(1815-1852)



WOOSONG UNIVERSITY
Kazakhstan

# Evolution Stages

1. **Physical Coding**
   - Early computers used switches, wires, or punched cards.
   - Direct physical representation of binary instructions.

2. **Symbolic Documentation**
   - Engineers wrote human-readable notes documenting the binary code.

3. **Symbolic Coding**
   - Introduction of mnemonics (e.g., ADD, MOV) → *Assembly language.*

4. **Translation & Execution**
   - Assemblers/compilers translate symbolic code into binary machine code.
   - Enables faster programming and reduces human error.

5. **Requires a Translator**
   - Translator (Assembler) = converts symbolic → binary automatically.



WOOSONG UNIVERSITY
Kazakhstan

# Lecture plan

- Machine languages at a glance
- The Hack machine language:
  - Symbolic version
  - Binary version
- Perspective

- (The assembler will be covered in lecture 6).



WOOSONG UNIVERSITY
Kazakhstan

# Typical Elements of a Machine Language

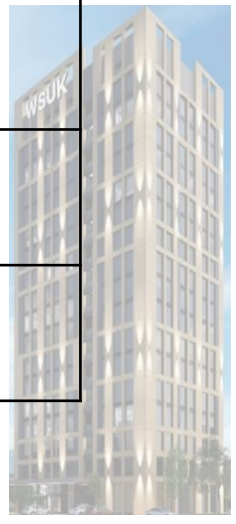
- **Instruction set:** small set of operations (load, store, add, jump…).
- **Registers:** fast storage (e.g., A, D, PC).
- **Memory access:** instructions reference RAM addresses.
- **Control flow:** conditional/unconditional jumps.



WOOSONG UNIVERSITY
Kazakhstan

# Typical Elements of a Machine Language

| Element | Description | Example (Hack Machine) |
|---|---|---|
| Instruction Set | Defines the basic operations the CPU can perform — arithmetic, logic, and control. | D = A + 1 → add A + 1, store result in D |
| Registers | Small, fast storage inside the CPU used to hold data or addresses. | A, D, and M (where M = RAM[A]) |
| Memory Access | Allows reading or writing data from RAM using the A register as an address pointer. | @17 → A = 17, then D = M → D = RAM[17] |
| Control Flow | Alters the order of program execution (conditional/unconditional jumps). | 0;JMP → Jump always<br>D;JGT → Jump if D > 0 |
| Constants & Symbols | Numeric values or labels used in instructions to represent data or addresses. | @VALUE or @LOOP (symbol replaced with address) |
| Computation (comp) | Performs arithmetic/logical operations using ALU. | D = D + A, M = D & A, D = !M |
| Destination (dest) | Specifies where to store the result (in D, A, M, or combinations). | MD = D + 1 → store in M and D |
| Jump (jmp) | Conditional branching based on computation result (e.g., zero, negative, positive). | D;JEQ → Jump if D == 0<br>D;JLT → Jump if D < 0 |



# Typical machine language commands (a small sample)

```
// In what follows R1,R2,R3 are registers, PC is program counter,
// and addr is some value.

 ADD R1,R2,R3      // R1 ← R2 + R3

 ADDI R1,R2,addr   // R1 ← R2 + addr

 AND R1,R1,R2      // R1 ← R1 and R2 (bit-wise)

 JMP addr          // PC ← addr

 JEQ R1,R2,addr    // IF R1 == R2 THEN PC ← addr ELSE PC++

LOAD R1, addr     // R1 ← RAM[addr]

STORE R1, addr    // RAM[addr] ← R1

NOP               // Do nothing

// Etc. – some 50-300 command variants
```



WOOSONG UNIVERSITY
Kazakhstan

# The Hack computer

A 16-bit machine consisting of the following elements:

- **Data memory: RAM** - an addressable sequence of registers storing data
- **Instruction memory: ROM** - an addressable sequence of registers storing the program code (instructions)
- **Registers:** D, A, M, where **M** stands for **RAM[A]** (the memory location pointed to by A)
- **Processing: ALU**, Performs arithmetic and logical operations on data.
- **Program counter: PC**, holds the address of the next instruction to excute.
- **Control:** ROM is preloaded with 16-bit instructions; execution starts at address 0 and proceeds in a fetch-execute cycle.
- **Instruction set:** Two instructions:
  - **A-instruction:** Used to load data/address into the A register.
  - **C-instruction:** Performs computations and controls data flow.



# The A-instruction (Address instruction)

```
@value        // A ← value
```

Where *value* is either a number (constant) or a symbol referring to some number (address).

Used for:

- Entering a constant value (`A = value`)

Coding example:

```
@17     // A = 17
D = A   // D = 17
```

- Selecting a `RAM` location (`register = RAM[A]`)

```
@17     // A = 17   (points to RAM[17])
D = M   // D = RAM[17]
```

Later

- Selecting a `ROM` location (`PC = A`)

```
@17     // A = 17 (target address)
JMP     // fetch the instruction
        // stored in ROM[17]
```



# The C-instruction (Compute Instruction) (first approximation)

*dest* = *x* + *y*

*dest* = *x* − *y*

*dest* = *x*

*dest* = 0

*dest* = 1

*dest* = −1

Inputs come from A, D, or M (where M = RAM[A]).

*x* = {A, D, M}

*y* = {A, D, M , 1}

*dest* = {A, D, M, MD, A, AM, AD, AMD, null}

Exercise: Implement the following tasks using Hack commands:

- Set `D` to `A-1`
- Set both `A` and `D` to `A + 1`
- Set `D` to `19`
- Set both `A` and `D` to `A + D`
- Set `RAM[5034]` to `D - 1`
- Set `RAM[53]` to `171`
- Add `1` to `RAM[7]`, and store the result in `D`.



# The C-instruction (first approximation)

```
dest = x + y
dest = x - y
dest = x
dest = 0
dest = 1
dest = -1
```

```
x = {A, D, M}
y = {A, D, M , 1}
dest = {A, D, M, MD, A, AM, AD, AMD, null}
```

MD= store in both D and M

Symbol table:

| | |
|---|---|
| j | 3012 |
| sum | 4500 |
| q | 3812 |
| arr | 20561 |

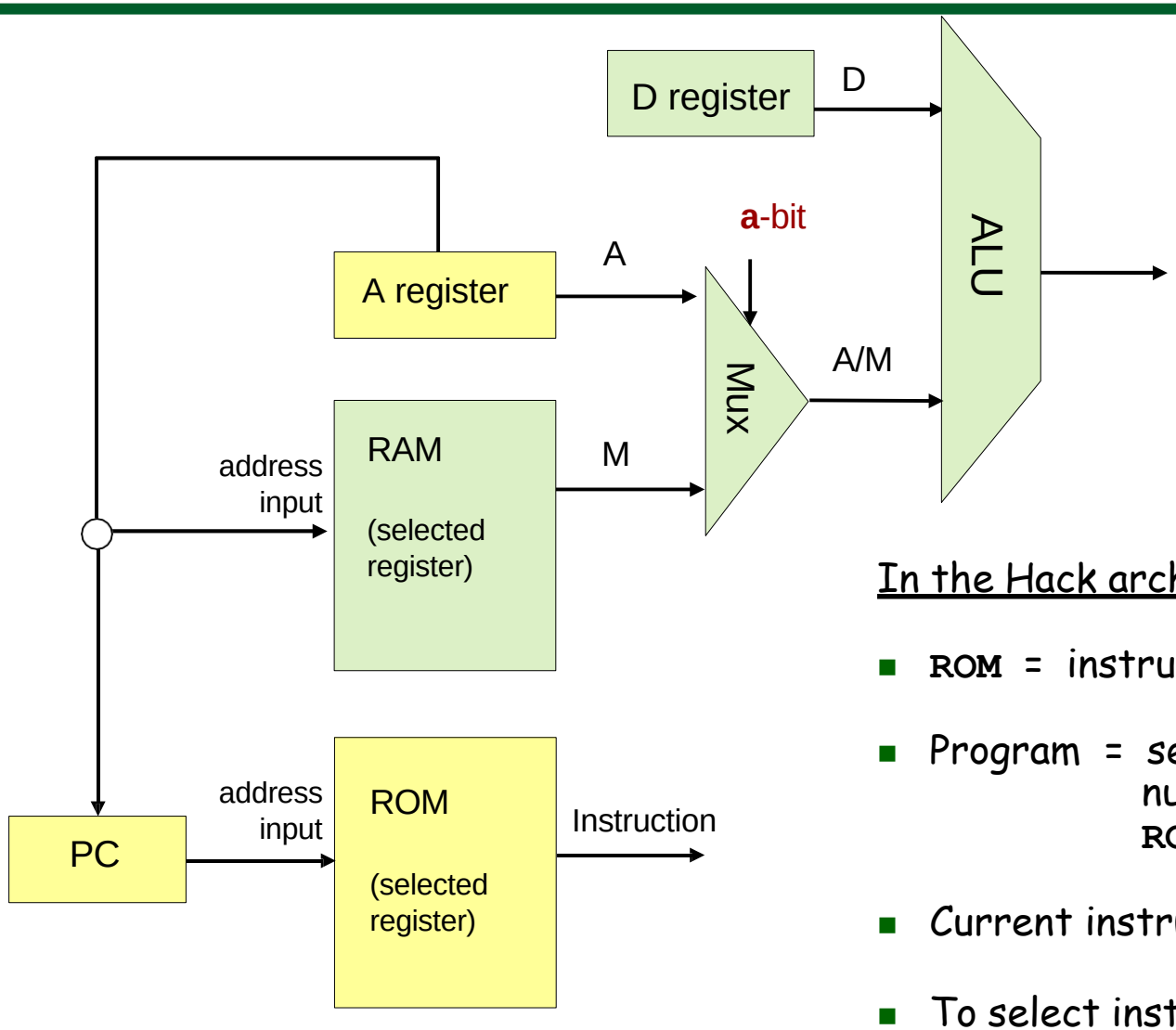
(All symbols and values are arbitrary examples)

Exercise: Implement the following tasks using Hack commands:

- `sum = 0`
- `j = j + 1`
- `q = sum + 12 - j`
- `arr[3] = -1`
- `arr[j] = 0`
- `arr[j] = 17`
- `etc.`



# Control (focus on the yellow chips only)




In the Hack architecture:

- **ROM** = instruction memory
- Program = sequence of 16-bit numbers, starting at **ROM[0]**
- Current instruction = **ROM[PC]**
- To select instruction *n* from the **ROM**, we set **A** to *n*, using the instruction **@n**

# Coding examples (practice)

Exercise: Implement the following tasks using Hack commands:

- goto 50
- if D==0 goto 112
- if D<9 goto 507
- if RAM[12] > 0 goto 50
- if sum>0 goto END
- if x[i]<=0 goto NEXT.

## Hack commands:

A-command: @value // set A to value

C-command: dest = comp ; jump // dest = and ;jump // are optional

Where:

comp = 0 , 1 , -1 , D , A , !D , !A , -D , -A , D+1 , A+1 , D-1, A-1 , D+A , D-A , A-D , D&A , D|A , M , !M , -M ,M+1, M-1 , D+M , D-M , M-D , D&M , D|M

dest = M , D , MD , A , AM , AD , AMD, or null

jump = JGT , JEQ , JGE , JLT , JNE , JLE , JMP, or null

In the command dest = comp; jump, the jump materialzes if (comp jump 0) is true. For example, in D=D+1,JLT, we jump if D+1 < 0.

## Hack convention:

- True is represented by -1
- False is represented by 0

because of 2's complement 16 bit : 0 and -1 (15)

Symbol table:

| | |
|---|---|
| sum | 2200 |
| x | 4000 |
| i | 6151 |
| END | 50 |
| NEXT | 120 |

(All symbols and values in are arbitrary examples)



WOOSONG UNIVERSITY
Kazakhstan

| High-Level Task | Hack Commands | Explanation |
|---|---|---|
| goto 50 | @50<br>0;JMP | Unconditional jump: sets A=50 and PC ← 50, so execution continues at ROM[50]. |
| if D == 0 goto 112 | @112<br>D;JEQ | Conditional jump: jumps to address 112 only if D == 0 (JEQ = Jump if Equal to Zero). |
| if D < 9 goto 507 | @9<br>D = D - A<br>@507<br>D;JLT | Hack compares only with 0, so we compute D−9. If result < 0, D;JLT jumps to 507. |
| if RAM[12] > 0 goto 50 | @12<br>D = M<br>@50<br>D;JGT | Loads RAM[12] into D. If D > 0, jump to 50 (JGT = Jump if Greater than Zero). |
| if sum > 0 goto END (sum=2200, END=50) | @2200<br>D = M<br>@50<br>D;JGT | Loads sum (RAM[2200]) into D. If D > 0, jump to label END (address 50). |
| if x[i] <= 0 goto NEXT (x=4000, i=6151, NEXT=120) | @6151<br>D = M<br>@4000<br>A = D + A<br>D = M<br>@120<br>D;JLE | Loads i, computes x+i, loads RAM[x[i]] into D. If D ≤ 0, jump to NEXT (address 120, JLE = Jump if Less or Equal). |



# IF logic – Hack style

High level:

```
if condition {
   code block 1}
else {
   code block 2}
code block 3
```

Hack:

```
D = not condition      // compute NOT of the condition
@IF_TRUE
D;JEQ                  // if NOT condition == 0 → means
                        condition was TRUE → jump to
                        IF_TRUE


// else part
code block 2           // executes if condition is FALSE
@END
0;JMP                  // skip IF_TRUE part and go to END


(IF_TRUE)
code block 1           // executes if condition is TRUE


(END)
code block 3           // runs always after if-else
```

Hack convention:

- True is represented by -1
- False is represented by 0

# WHILE logic – Hack style

High level:

```
while condition {
    code block 1
}
Code block 2
```

Hack:

```
(LOOP)                      // label marking the
                            start of the loop

   D = not condition       // evaluate the
                            condition and store
                            the result in D

   @END

   D;JEQ                    // if condition is
                            false (D == 0), jump
                            to END

   code block 1             // loop body

   @LOOP

   0;JMP                     // go back to start
                            to check condition
                            again

(END)

   code block 2             // executes after
                            loop finishes
```

Hack convention:

- True is represented by -1
- False is represented by 0

# WHILE logic – Hack style

High level:

```
value = 0
while (value < 4)
    {
    value = value
    + 1
}
```

Hack:

```
@0
D=A
@value
M=D              // value = 0

(LOOP)
   @value
   D=M           // D = value
   @4
   D=D-A         // D = value - 4
   @END
   D;JGE         // if value >= 4, jump to
                    END (exit
                    loop)
   @value
   M=M+1         // value = value + 1

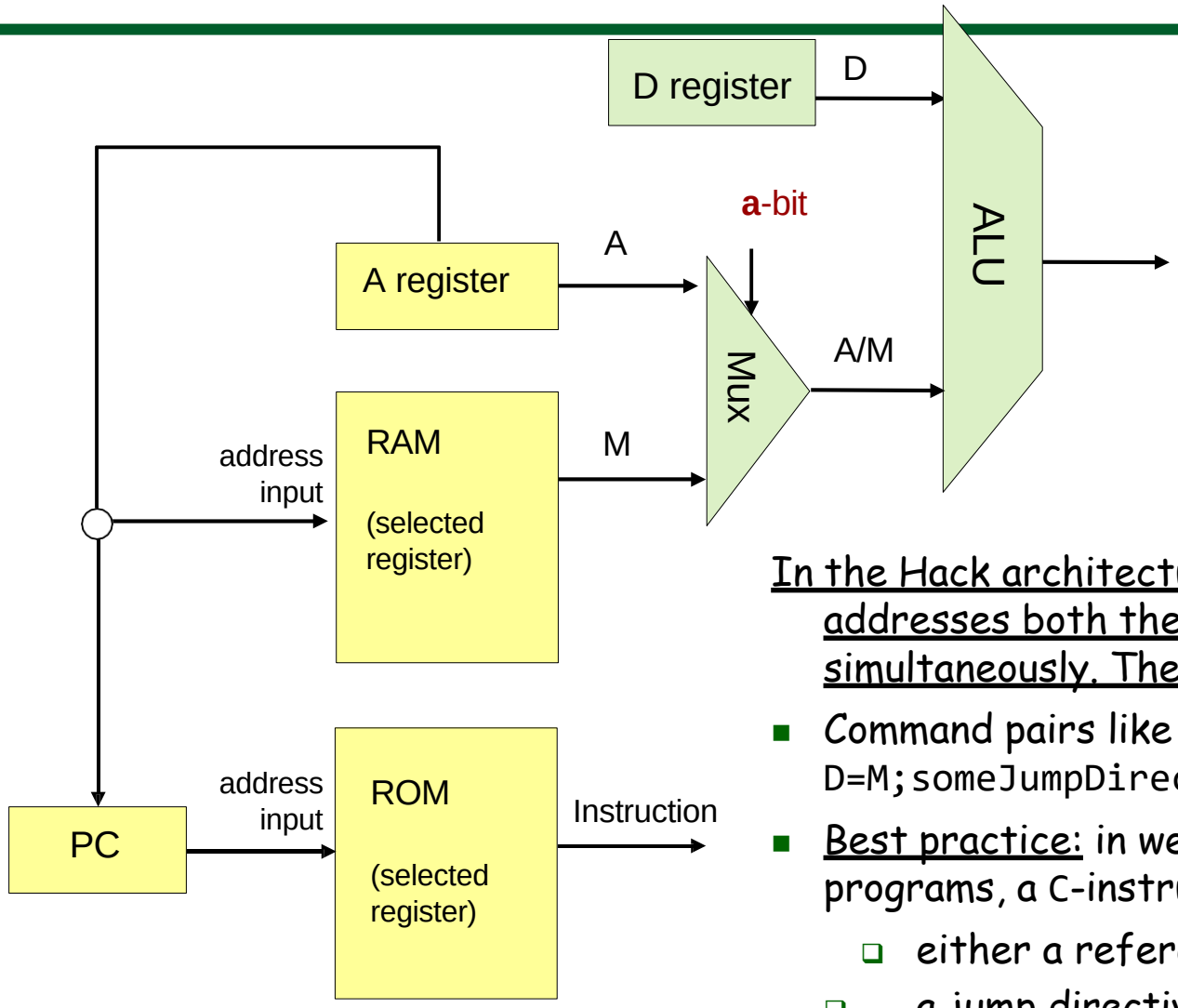
   @LOOP
   0;JMP         // repeat

(END)
   // done
```

Hack convention:

- True is represented by -1
- False is represented by 0

# Side note (focus on the yellow chip parts only)




In the Hack architecture, the A register addresses both the RAM and the ROM, simultaneously. Therefore:

- Command pairs like `@addr` followed by `D=M;someJumpDirective` make no sense
- Best practice: in well-written Hack programs, a C-instruction should contain
  - either a reference to `M`, or
  - a jump directive,

  but not both.

# Complete program example

C language code:

```
// Adds 1+...+100.
 into i = 1;
 into sum = 0;
 while (i <= 100){
    sum += i;
    i++;
 }
```

Hack assembly code:

```
// Adds 1+...+100.
       @i      // i refers to some RAM location
       M=1     // i=1
       @sum    // sum refers to some RAM location
       M=0     // sum=0
(LOOP)
       @i
       D=M     // D = i
       @100
       D=D-A   // D = i - 100
       @END
       D;JGT   // If (i-100) > 0 goto END
       @i
       D=M     // D = i
       @sum
       M=D+M   // sum += i
       @i
       M=M+1   // i++
       @LOOP
       0;JMP   // Got LOOP
 (END)
       @END
       0;JMP   // Infinite loop
```

Hack assembly convention:

- Variables: lower-case
- Labels: upper-case
- Commands: upper-case

Demo
CPU emulator



# Symbols in Hack assembly programs

Symbols created by Hack programmers and code generators:

Predefined symbols:

- I/O pointers: The symbols `SCREEN` and `KBD` are "automatically" predefined to refer to RAM addresses 16384 and 24576, respectively (base addresses of the Hack platform's *screen* and *keyboard* memory maps)
- Virtual registers: covered in future lectures.
- VM control registers: covered in future lectures.

```
// Typical symbolic
// Hack code, meaning
// not important
   @R0
   D=M
   @INFINITE_LOOP
   D;JLE
   @counter
   M=D
   @SCREEN
   D=A
   @addr
   M=D
(LOOP)
   @addr
   A=M
   M=-1
   @addr
   D=M
   @32
   D=D+A
   @addr
   M=D
   @counter
   MD=M-1
   @LOOP
   D;JGT
(INFINITE_LOOP)
   @INFINITE_LOOP
   0;JMP
```

Q: Who does all the "automatic" assignments of symbols to RAM addresses?

A: The *assembler*, which is the program that translates symbolic Hack programs into binary Hack program. As part of the translation process, the symbols are resolved to RAM addresses. (more about this in future lectures)

WOOSONG UNIVERSITY
Kazakhstan

- **Label symbols**: Used to label destinations of goto commands. Declared by the pseudo command **(XXX)** . This directive defines the symbol **XXX** to refer to the instruction memory location holding the next command in the program (within the program, **XXX** is called "label")

- **Variable symbols**: Any user-defined symbol **xxx** appearing in an assembly program that is not defined elsewhere using the **(xxx)** directive is treated as a variable, and is "automatically" assigned a unique RAM address, starting at RAM address 16

- By convention, Hack programmers use lower-case and upper-case letters for variable names and labels, respectively.



WOOSONG UNIVERSITY
Kazakhstan

# Perspective

- Hack is a simple machine language
- User friendly syntax: `D=D+A` instead of `ADD D,D,A`
- Hack is a "½-address machine": any operation that needs to operate on the RAM must be specified using two commands: an **A**-command to address the RAM, and a subsequent C-command to operate on it
- A Macro-language can be easily developed
- A Hack assembler is needed and will be discusses and developed later in the course.

